

PRODUCT RESEARCH ASSISTANT

A Deep-Dive Technical Blueprint, API Reference, and Setup Guide

Created By: Antigravity AI Assistant
OS Platform: Windows Developer Suite
Date: August 2026
Version: 2.1.0

1. Executive Summary

The Product Research Assistant is an enterprise-grade agentic workflow tailored to solve the friction of searching, parsing, and comparing consumer electronics from complex mock catalogs. It leverages large language model intelligence (specifically Google's Gemini models) to dynamically interact with structured data, generating summaries and comparisons tailored to specific user needs and price ranges.

By packaging the database connection as a Model Context Protocol (MCP) tool server and the agent runtime inside a FastAPI framework, the application ensures stateless, scalable execution on modern serverless platforms like Google Cloud Run.

2. System Architecture Blueprint

The system's modular architecture separates the client interface, model processing pipelines, and data layers:

- **Dashboard Client (React 18 + Tailwind CSS):** Provides a high-fidelity dashboard. Features a persistent chat module on the left side and a dynamic results visualization panel on the right side. The UI coordinates tab state transitions between general search results, side-by-side spec comparison tables, and the Best Choice recommendation card.
- **API Gateway & Static Server (FastAPI):** Implements stateless CORS headers, binds static assets generated by Vite, and handles POST-based session states. The `/run` endpoint processes incoming request vectors, maps sessions, and streams the output directly to the user.`
- **Agent Orchestration Layer (Google ADK SDK):** Utilizes the Gemini developer framework to define instructions, parse tool calls, handle exceptions, and compose recommendations.
- **MCP Database Layer (FastMCP SSE):** Acts as a secure, network-bound server that exports SQLite-equivalent Python tools. Because of Cloud Run networking characteristics, it runs without DNS rebinding protection to enable secure local and remote socket connections.

3. Core Optimizations and Upgrades

During Phase 2, the application underwent significant performance tuning and feature expansion to transition from a prototype laptop search engine to a universal consumer electronics research suite:

3.1. Catalog Expansion & Schema Augmentation

The database catalog inside ``server.py`` was increased to 26 products across five major domains: Laptops, Tablets, Smartphones, Audio Devices, and Monitors. Additionally, a `url` schema field was appended to the Pydantic data model. This field maps each item to its official manufacturer product page.

3.2. Search Logic Category Parsing

To allow flexible and natural search queries, we updated the category matching module inside `search_products` with the following category mapping structure:

```
category_mapping = { "laptop": ["laptop", "notebook", "computer", "programming", "coder",  
"developer"], "tablet": ["tablet", "tab", "ipad"], "smartphone": ["phone", "smartphone",  
"iphone", "pixel", "galaxy", "mobile"], "audio": ["audio", "headphones", "earbuds", "noise  
canceling", "anc", "music"], "monitor": ["monitor", "display", "screen"] }
```

This category intent dictionary ensures that generic terms are matched against the product category specs, allowing the model to instantly find appropriate items even if specific product names are omitted from the query.

3.3. Switching to Gemini 3.5 Flash Lite

Initially, the system was configured with ``gemini-3.6-flash``, which experienced high request latency (averaging over 3 minutes) due to deprecations, leading to 504 Gateway errors. By updating the model configuration in ``agent.py`` to use ``gemini-3.5-flash-lite``, we successfully reduced average execution latency to **under 15 seconds** while maintaining tool-calling accuracy.

4. Installation & Local Running Guide

Follow these concrete setup commands to build and run the services in a Windows local environment:

Step 1: Build the React Application

In the frontend folder, run npm install and Vite build to populate static assets into the backend folder:

```
cd frontend cmd /c npm install cmd /c npm run build
```

Step 2: Launch the MCP Server

Boot the MCP server on port 8081 using the backend virtual environment's interpreter:

```
cd mcp_server ..\backend\.venv\Scripts\python server.py 8081
```

Step 3: Boot the Backend Web Application

Configure the `.env` parameters inside `backend/.env` and launch the FastAPI server:

```
cd backend edit .env file: GEMINI_API_KEY=your_gemini_api_key  
PRODUCT_MCP_URL=http://localhost:8081/sse PORT=8080 .venv\Scripts\python main.py
```

5. Google Cloud Run Deployment Guide

Deploy the containerized Docker containers to Cloud Run using Google Cloud SDK:

```
# Submit and deploy the MCP Server gcloud builds submit --tag  
us-central1-docker.pkg.dev/[PROJECT_ID]/mcp-repo/product-mcp-server gcloud run deploy  
product-mcp-server --image  
us-central1-docker.pkg.dev/[PROJECT_ID]/mcp-repo/product-mcp-server --platform managed  
--region us-central1 --allow-unauthenticated --port 8081 --quiet # Submit and deploy the  
Backend Server gcloud builds submit --tag  
us-central1-docker.pkg.dev/[PROJECT_ID]/adk-repo/product-research-backend gcloud run deploy  
product-research-backend --image  
us-central1-docker.pkg.dev/[PROJECT_ID]/adk-repo/product-research-backend --platform  
managed --region us-central1 --allow-unauthenticated --set-env-vars PRODUCT_MCP_URL=https://  
/product-mcp-server-450158463415.us-central1.run.app/sse,GEMINI_API_KEY=...,GOOGLE_API_USE_  
CLIENT_CERTIFICATE=false --port 8080 --quiet
```